

SKILL EXPERIMENT 19

Name: K ANSHU

Roll No: 2520030440

Section: 8

Title

Implementation of Signal Handling, SIGINT Forwarding and Foreground Job Control

Aim

To register signal handlers, configure signal actions, handle interrupts, manage signal context, verify stability, and test signal events.

To capture SIGINT, forward signals, protect the shell process, terminate foreground jobs, update job states, and test interrupt handling.

Theory

Signal handlers allow a process to respond to specific signals generated by the operating system or terminal. Signal actions can be configured to define how a process responds when a particular signal is received.

SIGINT is commonly generated by the terminal when an interrupt operation is performed. In a shell environment, the shell must protect itself while ensuring that the appropriate foreground process or process group receives the interrupt signal.

After a foreground job is interrupted or terminated, its state must be updated in the job table. Proper signal handling and signal context management help maintain stable shell operation while allowing foreground jobs to respond correctly to interrupt events.

Algorithm

Part A – Signal Handler and Signal Actions

1. Start the program.
2. Initialize the signal handling mechanism.
3. Register the required signal handlers.
4. Configure the required signal actions.
5. Identify the signals that need to be handled.

6. Receive a signal event.
7. Invoke the corresponding signal handler.
8. Manage the signal context during handling.
9. Display the received signal information.
10. Verify that the process continues to operate correctly.
11. Test the configured signal events.
12. Stop the signal handling operation.

Part B – SIGINT and Foreground Job Handling

1. Start the shell or job-control operation.
2. Initialize the job table.
3. Identify the current foreground job.
4. Protect the shell process from unintended SIGINT termination.
5. Capture the SIGINT signal generated by the terminal.
6. Identify the foreground process or process group.
7. Forward SIGINT to the appropriate foreground job.
8. Allow the foreground job to terminate or change its state.
9. Wait for the updated job status.
10. Update the job state in the job table.
11. Verify that the shell process remains active.
12. Test interrupt handling with a foreground job.
13. Display the updated job information.
14. Stop the program.

CODE

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <signal.h>
```

```
#include <sys/wait.h>
```

```
pid_t foreground_job;
```

```
void shell_handler(int sig) {
```

```
    printf("\nShell received SIGINT.\n");
```

```
    printf("Forwarding SIGINT to foreground job...\n");
```

```
    if (foreground_job > 0)

        kill(foreground_job, SIGINT);

}


void job_handler(int sig) {

    printf("Foreground job received SIGINT.\n");

    printf("Foreground job is terminating...\n");

    _exit(0);

}


int main() {

    signal(SIGINT, shell_handler);


    printf("Shell signal handler registered.\n");

    printf("Starting foreground job...\n");


    foreground_job = fork();


    if (foreground_job == -1) {

        perror("fork");

        return 1;

    }


    if (foreground_job == 0) {
```

```
    signal(SIGINT, job_handler);

    printf("Foreground Job | PID = %d | State = Running\n",
        getpid());

    printf("Press Ctrl+C to send SIGINT.\n");

    while (1)
        pause();
}

printf("Shell is protected from direct termination.\n");

waitpid(foreground_job, NULL, 0);

printf("\nForeground job terminated.\n");
printf("Job State Updated: Terminated\n");
printf("Signal handling test completed successfully.\n");

return 0;
}
```

COMPILATION

```
hp@DESKTOP-35T9S8P:~$ nano exp4.c
hp@DESKTOP-35T9S8P:~$ gcc exp4.c -o exp4
hp@DESKTOP-35T9S8P:~$ ./exp4
Shell signal handler registered.
Starting foreground job...
Shell is protected from direct termination.
Foreground Job | PID = 504 | State = Running
Press Ctrl+C to send SIGINT.
^CForeground job received SIGINT.
Foreground job is terminating...

Shell received SIGINT.
Forwarding SIGINT to foreground job...

Foreground job terminated.
Job State Updated: Terminated
Signal handling test completed successfully.
hp@DESKTOP-35T9S8P:~$ |
```

Result

Signal handlers and signal actions were configured successfully. Signal events were handled and the signal context was managed correctly. SIGINT was captured and forwarded to the foreground job while protecting the shell process. The foreground job state was updated and interrupt handling was verified successfully.